

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
SCUOLA DI INGEGNERIA E ARCHITETTURA (SEDE DI BOLOGNA)
Anno Accademico 2024/2025

**RELAZIONE DI FINE TIROCINIO CURRICULARE/TIROCINIO PER
TESI**

svolto dallo studente:
Matteo Lombardi (matricola nr. 0001071217)

iscritto al Corso di Studio in:
Ingegneria Informatica (codice CdS: 9254)

presso:
Apra S.p.A.

sul seguente argomento:
Estrazione automatizzata di informazioni da immagini tramite l'uso di large
language models multimodali

Studente:

Matteo Lombardi (firma)

Referente struttura ospitante:

Christian Lillini, [ruolo all'interno alla struttura]

Christian Lillini
(firma per approvazione della relazione finale)

Indice

1	Introduzione	3
2	Corpo	5
2.1	Definizione degli obiettivi	5
2.2	Fase di formazione	6
2.3	Verifica della fattibilità del progetto	7
2.4	Progettazione dell'architettura di sistema	8
2.5	Estrazione delle informazioni da immagini	10
2.6	Matching tra scontrini e ricevute di pagamento	11
2.7	Testing, ottimizzazione e risultati	12
2.8	Costi di esecuzione e scalabilità	20
3	Conclusione	23

1 Introduzione

Questo documento presenta la mia esperienza di tirocinio presso Apra S.p.A., un'azienda facente parte del Var Group, punto di riferimento nazionale per il settore ICT, specializzata nello sviluppo di software finalizzato alla digitalizzazione delle imprese. In totale il tirocinio ha avuto una durata di 160 ore, svolte durante un mese di lavoro full time, con l'obiettivo principale di insegnare e applicare large language models (LLM) multimodali per l'estrazione di informazioni da immagini.

Nello specifico, dopo un periodo iniziale di formazione sul linguaggio Python per la programmazione orientata agli oggetti e sull'integrazione di modelli di intelligenza artificiale, mi è stata affidata la completa realizzazione di un progetto utile al miglioramento dell'automazione di alcuni processi aziendali. Tale progetto è nato dall'esigenza di conformarsi alle disposizioni introdotte dalla Legge di Bilancio 2025, con la quale, a partire dal 1° Gennaio 2025, è entrato in vigore uno specifico vincolo per quanto riguarda la deducibilità delle spese aziendali: *A decorrere dal 1° gennaio 2025, le spese sostenute per trasferte, vitto, alloggio e rappresentanza sono deducibili dal reddito d'impresa e di lavoro autonomo esclusivamente se effettuate mediante strumenti di pagamento tracciabili, quali bonifici bancari o postali, carte di credito, di debito o prepagate, altri strumenti previsti dall'art. 23 del D.Lgs. n. 241/1997.*¹ Per i primi mesi del 2025, in Var Group, tale processo di verifica del metodo di pagamento veniva svolto manualmente, con la necessità di dover controllare migliaia di immagini di scontrini al mese e il conseguente impiego di settimane di lavoro da parte di dipendenti dedicati. Una volta appreso tale problema e carpite le mie esigenze e preferenze di apprendimento, il mio responsabile ha deciso di coinvolgermi in questo progetto di ricerca finalizzato alla realizzazione di un sistema automatizzato capace di ridurre o sostituire questa procedura manuale.

L'attività di tirocinio è stata quindi suddivisa in diverse fasi, partendo dal lavoro di apprendimento di Python e dell'utilizzo di LLM ho poi progettato l'architettura di sistema, realizzato i moduli per l'estrazione e l'analisi delle informazioni, e infine testato e ottimizzato l'algoritmo su un insieme di immagini fornito dall'azienda. Uno degli aspetti che ha richiesto maggiore attenzione è stata la gestione di set di immagini caratterizzate da una grande variabilità nella struttura dei documenti, nel formato e nella qualità visiva.

Questa esperienza mi ha permesso di comprendere a fondo il funzionamento di alcuni processi aziendali dall'interno, e, curando in prima persona lo sviluppo completo di un progetto, sono riuscito a ottenere una panoramica dettagliata di ogni fase di un lavoro di questo tipo: dall'analisi del problema alla progettazione,

¹Legge 30 dicembre 2024, n. 207 – Legge di Bilancio 2025.

implementazione e ottimizzazione. Nella pratica ho acquisito competenza sull'utilizzo di Python per la programmazione orientata agli oggetti, sull'integrazione di LLM multimodali e sulla gestione degli input e output di questi modelli. Nel proseguimento del documento verranno affrontati più nel dettaglio i seguenti aspetti: l'esperienza di tirocinio nel suo complesso, il processo di sviluppo, le tecnologie utilizzate e gli obiettivi raggiunti.

2 Corpo

2.1 Definizione degli obiettivi

Il primo giorno di tirocinio è stato dedicato alla presentazione, da parte del mio responsabile Christian, dell'azienda, del mio ufficio e del progetto a cui avrei lavorato. Mi sono state presentate tutte le persone con cui sarei stato a stretto contatto per tutta la durata del tirocinio, compresi coloro che si occupavano dell'analisi manuale delle immagini e del mantenimento del gestionale preposto alla memorizzazione delle informazioni relative alle spese deducibili dall'azienda.

Subito dopo, Christian mi ha illustrato in dettaglio tutti gli obiettivi del mio tirocinio, fornendomi una panoramica temporale del mio mese di lavoro e delle attività su cui mi sarei concentrato. La struttura era dunque la seguente:

- Prima settimana: formazione approfondita su Python per la programmazione ad oggetti, applicazioni con AI e LLM.
- Restanti settimane: analisi, progettazione, sviluppo, testing e consegna del progetto.

Mi è stata descritta nel dettaglio la Legge di Bilancio 2025 che, come anticipato nell'introduzione, imponeva all'azienda, oltre alla consueta raccolta delle immagini relative alle spese detraibili (come avveniva fino al 31 dicembre 2024), anche la verifica della presenza, su ciascuno scontrino, di una dicitura che certificasse l'utilizzo di uno strumento di pagamento tracciabile. La necessità di creare un software capace di svolgere questo lavoro in maniera automatizzata ha portato, già in fase di presentazione e discussione, all'individuazione dei principali problemi che avrei dovuto affrontare. Uno dei più rilevanti riguardava la scelta della tecnologia da utilizzare per lo sviluppo dell'applicativo e l'estrazione dei dati dalle immagini. Su questo aspetto mi è stata concessa ampia autonomia, il che mi ha permesso di comprendere come, nei progetti innovativi all'interno di un contesto aziendale, il raggiungimento degli obiettivi sia spesso considerato più importante del vincolo sugli specifici strumenti utilizzati. Mi sono state comunque fornite alcune indicazioni e raccomandazioni utili a riguardo, come il suggerimento di utilizzare Python come linguaggio di programmazione, per via della sua ampia compatibilità con i principali modelli di intelligenza artificiale, e di fare uso di large language models multimodali per la lettura e l'estrazione delle informazioni dalle immagini. Un altro problema che mi è stato presentato riguarda l'eterogeneità dei documenti in input, che possono essere PDF, scansioni o immagini acquisite tramite smartphone. Molti di questi file vengono inoltre compressi durante l'invio a causa delle impostazioni del software di gestione, compromettendo in alcuni casi la qualità delle informazioni e causando potenziali errori di lettura. Infine, il formato degli scontrini varia notevolmente da un esercente all'altro,

portando a frequenti situazioni in cui la modalità di pagamento risulta incerta o difficilmente identificabile.

A seguito di un veloce sguardo ad alcuni documenti di input, sono stati definiti i seguenti obiettivi di progetto:

- Automatizzare l'analisi delle immagini ai fini dell'identificazione delle spese aziendali detraibili, garantendo la maggiore accuratezza possibile.
- Garantire la possibilità di eseguire l'applicativo sia localmente che tramite chiamata remota fungendo da microservizio.
- Verificare l'efficacia del programma effettuando test su un ampio dataset reale fornito dall'azienda.

2.2 Fase di formazione

Il lavoro è quindi iniziato con una fase di formazione. Anche in questo caso mi è stata lasciata piena libertà nella scelta delle fonti da utilizzare, con il suggerimento di fare riferimento alla piattaforma devACADEMY, affiliata all'azienda, che offre percorsi completi su temi rilevanti per il progetto.

Qui ho approfondito gli aspetti fondamentali e avanzati della programmazione in Python e dell'integrazione di sistemi di intelligenza artificiale all'interno delle proprie applicazioni tramite i corsi "Programmazione a oggetti in Python" e "Applicazioni con AI". Il primo affrontava le basi di Python, che già conoscevo per via di alcuni progetti del liceo e personali, proseguendo poi per la definizione approfondita delle classi, degli oggetti, del concetto di ereditarietà e di alcuni aspetti peculiari del linguaggio di programmazione Python. Il corso aveva una struttura molto chiara e ogni concetto è stato trattato con grande attenzione. Il tutto si è concluso con lo sviluppo di un progetto finale che fungeva da esempio, il quale mi ha permesso di consolidare le nozioni apprese comprendendone l'applicazione pratica. Una volta messi in pratica autonomamente i concetti acquisiti, soprattutto per verificarne il funzionamento in casi d'uso specifici, ho iniziato con il secondo corso, di durata più breve, incentrato sull'utilizzo di alcuni large language models tramite Python.

Nello specifico sono state affrontate due tipologie di interazione con LLM: la prima prevedeva l'esecuzione dei modelli direttamente sul proprio computer, mentre la seconda faceva uso di API per accedervi da remoto. Per l'esecuzione locale si è fatto uso di Ollama, uno strumento open source che consente di eseguire modelli linguistici sul proprio elaboratore. In questo modo si ottengono diversi vantaggi:

- Privacy: i dati non viaggiano in rete per raggiungere server esterni.

- Assenza di costi: una volta scaricato il modello si è pronti per l'utilizzo, senza dover pagare per l'utilizzo come accade con la maggior parte delle API cloud.
- Accessibilità offline e bassa latenza: può essere utilizzato anche offline e non vi sono ritardi dovuti alla trasmissione dei dati nella rete.
- Maggiore controllo: vi è la possibilità di personalizzare completamente il modello modificandone il comportamento per adattarlo al contesto di utilizzo.

Tuttavia, questa modalità comporta anche alcuni svantaggi, come l'elevato consumo di risorse hardware (soprattutto RAM e GPU), il fatto che i modelli eseguibili localmente siano spesso meno potenti rispetto a quelli commerciali, e l'assenza di aggiornamenti automatici. Il corso includeva l'insegnamento della configurazione e dell'utilizzo di entrambe le modalità così da avere la possibilità di scegliere l'alternativa più adatta al nostro caso. Infine è stato affrontato l'utilizzo del framework LangChain per la gestione di prompt complessi e catene di elaborazione.

2.3 Verifica della fattibilità del progetto

Terminata la fase di formazione, occorre definire la struttura base del sistema e le principali tecnologie utilizzate. Come prima cosa ho deciso di effettuare alcuni test per verificare la fattibilità del progetto e, in caso di esito positivo, individuare le tecnologie più adatte. Sebbene l'idea iniziale, suggerita anche dal mio responsabile, fosse quella di utilizzare LLM per l'estrazione delle informazioni, data la necessità di comprendere non solo il testo ma anche il contesto generale dell'immagine, ho comunque voluto sperimentare il funzionamento di alcuni strumenti OCR, che consentono la conversione di immagini contenenti testo in testo digitale modificabile. L'obiettivo era quello di comprendere se l'estrazione del testo fosse più precisa e, in tal caso, aggiungerla al prompt da fornire al modello LLM per facilitarne l'analisi.

Ho deciso di testare Tesseract OCR, un motore open source mantenuto da Google per il riconoscimento ottico dei caratteri, attualmente tra i più utilizzati. Tuttavia, fornendo in input alcune immagini di scontrini di media qualità, il motore non è riuscito a estrarre informazioni corrette e ben strutturate, commettendo diversi errori nel riconoscimento di lettere e cifre. Si tratta di un aspetto critico per il nostro progetto, in cui la precisione dell'analisi è fondamentale, per questo motivo ho scelto di abbandonare la strada degli OCR, con la speranza di trovare un LLM che, con dei prompt efficaci, sarebbe stato in grado di analizzare correttamente le immagini. Ho iniziato utilizzando LLM in locale tramite Ollama, come

appreso durante la settimana di formazione, così da evitare costi di utilizzo nel caso avessi individuato un modello adatto. Ho quindi scaricato alcuni dei modelli multimodali più noti, tra cui LLava, Donut, Layout LM e Deepseek-VL. I primi tre hanno mostrato notevoli imprecisioni, commettendo errori simili a quelli di Tesseract su immagini compresse, mentre Deepseek-VL non è riuscito a fornire risposte a causa delle troppe risorse richieste. A questo punto ho dovuto scegliere se proseguire con i modelli locali, richiedendo all'azienda una macchina più potente, oppure iniziare a utilizzare modelli più performanti accessibili tramite API. Ho optato per questa seconda alternativa, per via dei vantaggi già evidenziati. Il passo successivo è stato quindi quello di richiedere al mio responsabile una API key per accedere a GPT-4o, l'LLM multimodale attualmente più indicato per questo tipo di attività. Una volta ricevuta la chiave API e analizzato le stesse immagini, ho notato immediatamente un netto miglioramento rispetto ai modelli testati in precedenza. L'output fornito sia da GPT-4o che da GPT-4o mini era corretto, non vi era nessun errore di lettura e le informazioni erano state interpretate correttamente, permettendomi di ricavare la tipologia di documento, l'importo e la data riportata sullo scontrino. La differenza tra i due modelli 4o e 4o-mini risiede nella potenza e nel costo: 4o è un modello dichiarato più potente, ma il suo costo per token utilizzato è circa 8 volte maggiore rispetto al modello mini. Era necessario dunque effettuare ulteriori test per discriminare se la differenza di potenza giustificasse il costo più elevato. Ho sviluppato dunque del codice ad hoc per il conteggio e il monitoraggio del numero complessivo di token utilizzati nelle diverse analisi, ed è emerso non solo che 4o rilevasse dati più corretti dallo stesso set di immagini, commettendo dunque un numero inferiore di errori, ma anche che 4o-mini generasse un numero di token circa 10 volte superiore rispetto al modello 4o, andando così a bilanciare la differenza di prezzo. La scelta è di conseguenza ricaduta sul modello più potente GPT-4o.

Questi test mi hanno quindi permesso di confermare la fattibilità del progetto e individuare la tecnologia da utilizzare per la fase di estrazione. A questo punto mancava solo da progettare l'architettura di sistema prima di iniziare a sviluppare il codice.

2.4 Progettazione dell'architettura di sistema

Durante questa fase è stata delineata l'infrastruttura generale del sistema con l'obiettivo di garantire modularità e facilità di manutenzione.

La struttura realizzata si basa su una pipeline di moduli distinti con responsabilità precise e ben definite:

- Modulo di acquisizione delle immagini.

Si occupa di raccogliere le immagini da analizzare, provenienti da archivi locali o da storage cloud come S3, implementando anche funzionalità di conversione di PDF in formati immagine supportati da GPT-4o.

- Modulo di parsing.

Utilizza il modello multimodale GPT-4o per analizzare le immagini e ricavarne informazioni chiave per stabilire se siano deducibili o meno, tra cui tipologia, data e ora, totale, categoria e altri attributi.

- Modulo di matching.

Ha il compito di correlare gli scontrini con le relative ricevute di pagamento elettronico basandosi sui dati estratti dal modulo di parsing. Per migliorare la classificazione degli scontrini sono state introdotte regole temporali e l'aggiunta di attributi che identificano l'evidenza di pagamento elettronico direttamente dall'analisi svolta dall'LLM.

- Modulo di testing.

Permette di confrontare gli output dell'applicativo con i dati annotati manualmente dagli addetti al lavoro di classificazione nei mesi precedenti allo sviluppo del progetto. In questo modo si ha la possibilità di ottenere una misura oggettiva dell'accuratezza del sistema, individuare gli errori ed eventualmente correggerli.

- Interfaccia API.

Consente di fungere da microservizio accessibile da remoto, come da requisiti iniziali di progetto. Questo consente l'integrazione con il sistema gestionale per la deducibilità dei costi aziendali.

L'architettura a pipeline è stata progettata appositamente per garantire pulizia nel codice e migliorarne la manutenibilità. Questa struttura permette infatti agli sviluppatori che lavoreranno al progetto di capirne velocemente l'organizzazione, identificare eventuali problemi in specifiche fasi del flusso di lavoro e isolarli per poter migliorare il sistema in modo mirato.

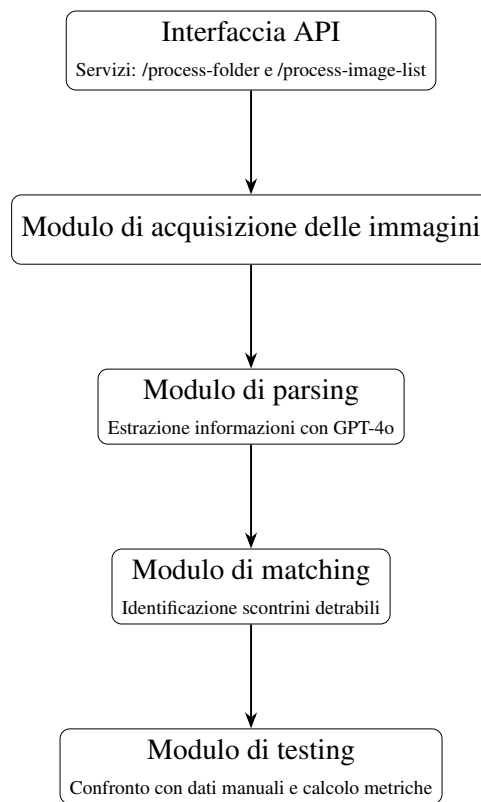


Figura 1: Architettura a pipeline del sistema

2.5 Estrazione delle informazioni da immagini

Una volta realizzata l'architettura di sistema sono passato alla fase di implementazione. Qui ho scelto di mantenere il progetto all'interno della piattaforma GitHub al fine di sfruttare tutti i vantaggi di un Version Control System, tra cui la possibilità di tenere traccia delle differenti versioni e poterci lavorare in parallelo con altri sviluppatori qualora ne fosse necessario. Dopo aver creato una nuova repository all'interno dell'account aziendale dedicato ai progetti di ricerca e sviluppo, ho iniziato lo sviluppo dei vari moduli architetturali, di cui descrivo di seguito le principali sfide affrontate e la struttura finale.

Il modulo principale, su cui si basa tutta l'estrazione delle informazioni dalle immagini fornite come input, è quello di parsing. Questo è un componente centrale dell'applicativo, poiché trasforma le immagini in una lista di oggetti Scontrino con le informazioni estratte. La fase di estrazione è attuata, come detto in precedenza, dal modello multimodale GPT-4o, al quale viene passato in input un'immagine per volta e un messaggio testuale, che in questo contesto viene chiamato prompt. L'output restituito è invece testo in formato JSON, che contiene una lista

semplice di tutti gli scontrini rilevati all'interno dell'immagine analizzata. Ho da subito compreso come una delle parti di maggiore rilevanza nella realizzazione di un applicativo con un'alta percentuale di correttezza fosse quella della scrittura di un prompt efficace. Questo doveva infatti spiegare in maniera accurata il contesto, composto da tipologie di scontrini molto diverse tra loro, le informazioni da estrarre e il formato di output da restituire. Una singola immagine poteva infatti contenere uno o più documenti fiscali, e ciascuno di questi poteva essere uno scontrino, una ricevuta di pagamento oppure entrambi sullo stesso foglio. Questa situazione non era semplice da descrivere al modello, per cui il prompt è stato sottoposto a diverse variazioni e perfezionamenti basati sui test, che vedremo più nel dettaglio nel capitolo di testing e ottimizzazione.

Ho scelto infine di restituire in output al modulo di parsing un oggetto `ReceiptsLists`, che mantiene al suo interno tre liste (lista di scontrini, lista di ricevute di pagamento e lista di documenti riconosciuti come "altro"), in modo da utilizzarlo come input nel modulo di matching elaborando ogni lista nella maniera appropriata.

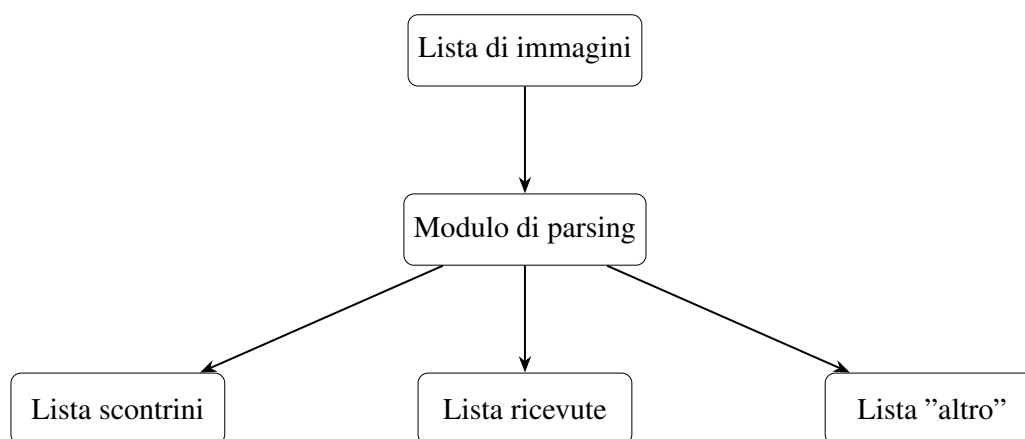


Figura 2: Diagramma a blocchi del modulo di parsing

2.6 Matching tra scontrini e ricevute di pagamento

Terminata la fase di estrazione delle informazioni, il passo successivo era quello di utilizzare questi dati per catalogare ogni scontrino come deducibile o meno. Il modulo di matching era stato pensato per svolgere questo compito. Per farlo, ho implementato una prima versione dell'algoritmo basata su una logica sequenziale e priva di ottimizzazioni avanzate.

In questo modo avrei potuto verificarne il corretto funzionamento, sviluppando una procedura che, ricevendo in input le tre liste ritornate dal modulo di parsing, svolgeva i seguenti passaggi:

1. Per ogni scontrino, dopo aver controllato che nessun campo al suo interno fosse stato rilevato dall'LLM come None, si cerca la corrispondente ricevuta di pagamento. Uno scontrino e una ricevuta di pagamento, per fare match, devono avere stessa data e stesso importo.
2. Se si rileva un match tra scontrino e ricevuta, i due elementi vengono associati e aggiunti alla lista dei costi deducibili; in caso contrario, lo scontrino viene archiviato tra gli elementi non deducibili.
3. I documenti la cui tipologia è stata rilevata come "altro" dal modello, quindi che non sono stati riconosciuti nè come scontrino nè come ricevuta di pagamento, vengono automaticamente aggiunti agli elementi non deducibili.

È facile intuire come l'output pensato per il modulo di matching è costituito da due liste, una con gli elementi deducibili e una con quelli non deducibili. Questo corrisponde anche al risultato finale previsto per il microservizio, sul quale sarà poi possibile intervenire effettuando test o generando un file CSV ad hoc per la rappresentazione dei dati.

L'obiettivo di questa prima fase di sviluppo era quello di implementare una struttura solida e funzionante con la quale poter realizzare un'ossatura generale del progetto e validare l'idea. L'approccio adottato è stato volutamente semplice ma modulare, consentendo di perfezionare il processo e aggiungere euristiche più avanzate nelle fasi successive.

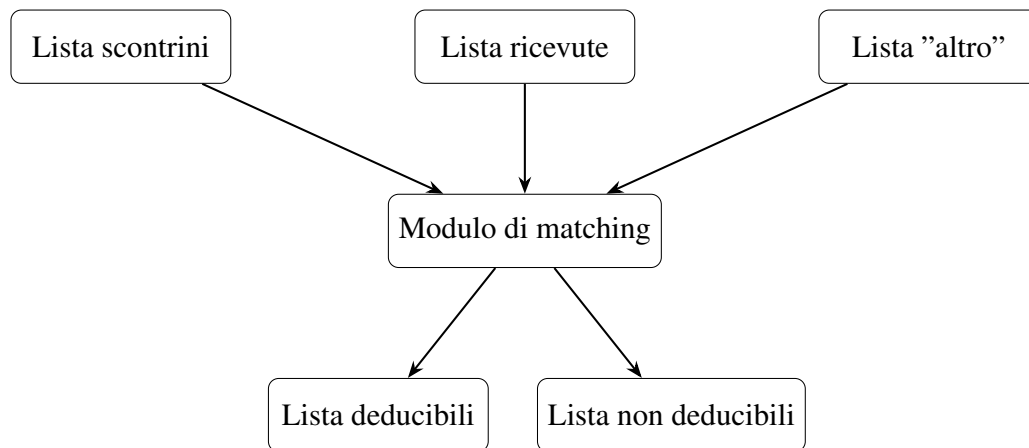


Figura 3: Diagramma a blocchi del modulo di matching

2.7 Testing, ottimizzazione e risultati

Dopo aver implementato la struttura base dei vari moduli e aver collegato la pipeline al fine di verificare che tutto funzionasse come previsto, ho deciso di dedicare

le ultime settimane di lavoro al testing del sistema su vari dataset, così da risolvere eventuali bug, ottimizzare l'algoritmo e avere metriche sull'efficacia del sistema.

Apra S.p.A. mi ha infatti messo a disposizione due dataset di immagini di scontrini ricevuti negli scorsi mesi, rispettivamente contenenti 25 e 600 immagini circa. In questo modo avrei utilizzato il primo insieme, caratterizzato da tipologie di immagini molto differenti tra loro, per svolgere test mirati al miglioramento generale dell'algoritmo, mentre il dataset più grande sarebbe stato utile per effettuare le ultime verifiche e ottenere le metriche. Ho deciso di operare in questa maniera perché ogni chiamata API comportava un costo effettivo, trattandosi di un modello a consumo e non a tariffa fissa, quindi l'utilizzo ripetuto del set più grande avrebbe implicato delle spese maggiori. Ho sviluppato dunque un algoritmo di testing che prevedeva il confronto tra i dati restituiti dal sistema con quelli raccolti a mano dai lavoratori, che mi erano stati forniti a questo scopo, facendomi comprendere l'importanza della fase di testing nello sviluppo di un progetto software.

Dopo aver aggiunto la possibilità di fornire in input una cartella di immagini provenienti dalla memoria locale, oltre a quella remota con S3, come emerso dalla revisione iniziale del progetto svolta da parte del responsabile, ho avviato la fase di test utilizzando il dataset di dimensioni più contenute. In questo modo ho risolto diversi bug che si presentavano in alcune casistiche particolari e inserito migliorie sia nel prompt che nel modulo di matching. Ad esempio, fino a quel momento la corrispondenza tra scontrino e ricevuta di pagamento veniva verificata esclusivamente confrontando data e importo totale. Questo portava all'individuazione di alcuni falsi positivi in cui lo scontrino veniva associato a una ricevuta con la stessa data e lo stesso totale, ma riferita in realtà a un altro pagamento. Per evitare errori di questo tipo ho deciso di modificare il prompt chiedendo al modello di restituire anche l'orario di pagamento nell'attributo 'datetime', e di controllare, nell'algoritmo di matching, la differenza tra gli orari di emissione di scontrino e ricevuta, tollerando uno scarto massimo di alcuni minuti.

Dopo l'implementazione di questa e altre migliorie l'algoritmo era pronto all'esecuzione del primo test con il dataset da 600 immagini, composto da 962 scontrini totali, di cui 853 detraibili e 109 non detraibili. Le metriche da calcolare erano le seguenti:

1. Accuratezza complessiva: percentuale di scontrini detraibili individuati correttamente sul totale.
2. Percentuale di errori per causa: controllo manuale di ciascuna analisi errata al fine di verificare la tipologia di errore. In questo modo viene facilitata l'ottimizzazione dell'algoritmo.
3. Tempo di esecuzione.

4. Costo totale di esecuzione.

Il primo test su larga scala ha individuato correttamente 700 scontrini detraibili sui 853 presenti, con un'accuratezza quindi dell'82%. La non presenza di falsi positivi, ovvero scontrini identificati come detraibili ma che in realtà non lo erano, ha rappresentato un ottimo risultato, in quanto il lavoro manuale da svolgere per gli addetti ai lavori sarebbe stato ridotto del 72.7%, ovvero da un paio di settimane a pochi giorni. Nonostante gli ottimi risultati ottenuti, analizzando la metrica sulla percentuale di errori per causa ho individuato alcuni margini di miglioramento. Ad esempio, ho rilevato come alcune tipologie specifiche di scontrini, come quelli relativi ai pedaggi autostradali, non venivano riconosciuti come tali e assegnati alla tipologia 'altro'. Così facendo, anche se il pagamento era avvenuto tramite strumento tracciabile e dunque deducibile come da Legge di Bilancio, questo veniva aggiunto automaticamente alla lista dei documenti non deducibili per via della struttura dell'algoritmo di matching spiegata in precedenza. Un altro errore ricorrente era dato dal fatto che molti scontrini non venivano rilasciati insieme a una ricevuta di pagamento, pur essendo pagati tramite strumento tracciabile, ma le informazioni relative al pagamento venivano riportate direttamente sullo scontrino stesso. Per evitare queste tipologie di falsi negativi ho deciso di modificare la struttura della classe Scontrino, aggiungendo l'attributo 'pagamento elettronico', un booleano che il modello assegnava come vero o falso in base a specifiche frasi presenti sul documento fiscale. Ciò però non bastava, in quanto la Legge di Bilancio dichiara solamente alcune particolari tipologie di scontrino come deducibili, ad esempio quelli relativi a ristorazione, hotel, autostrada e parcheggio. Per aggiungere questo controllo ho introdotto l'attributo 'categoria' e attuato di conseguenza alcuni cambiamenti sul prompt da inviare in input al modello multimodale, di cui riporto di seguito la versione finale.

```
1 prompt = (  
2     f'''  
3     Questa immagine contiene uno o più documenti (fogli),  
4     ↳ ciascuno dei quali può contenere uno o più elementi  
5     ↳ distinti: uno scontrino, una ricevuta POS, entrambi  
6     ↳ insieme nello stesso foglio, oppure un altro tipo di  
7     ↳ documento/non riconoscibile.  
8     Estrai tutti gli elementi riconoscibili presenti  
9     ↳ nell'immagine (se più elementi si trovano nello  
10    ↳ stesso foglio ritornali tutti) e restituisci il  
11    ↳ risultato come una lista JSON di oggetti, ciascuno  
12    ↳ con la seguente struttura:  
13    - type: "scontrino", "ricevuta pos" oppure "altro" (usa  
14    ↳ "scontrino" e "ricevuta pos" solo se sei sicuro del  
15    ↳ tipo, altrimenti "altro" se non rientra nelle  
16    ↳ categorie precedenti (es. pedaggio, parcheggio, ecc.)  
17    ↳ o non è riconoscibile)
```

```

6      - datetime: nel formato "dd-mm-YYYY HH:MM" (il documento
      ↪ mostra la data in formato "dd-mm-YYYY", "dd/mm/YYYY"
      ↪ oppure "dd/mm/YY", e l'ora nel formato "HH:MM"; tieni
      ↪ presente che l'anno corrente è il {current_year}; usa
      ↪ null se non rilevabile)
7      - total: numero decimale con il punto come separatore
      ↪ decimale (usa null se non rilevabile)
8      - category: una delle seguenti categorie in base al
      ↪ contenuto del documento: "autostrada", "parcheggio",
      ↪ "permesso ztl", "trasporti pubblici", "hotel",
      ↪ "ristorazione", "rapporto d'intervento" oppure
      ↪ "altro" (usa "altro" se l'elemento non rientra nelle
      ↪ categorie precedenti o non è riconoscibile)
9      - is_electronic_payment: booleano che indica se il
      ↪ pagamento è stato di tipo elettronico (es. termini
      ↪ come "pagamento elettronico", "modalità di pagamento
      ↪ carta/carta di credito/bancomat", "identificativo
      ↪ pagamento", "id transazione", "telepass", cifre
      ↪ finali della carta ecc.).
10     Per favore, rispondi fornendo solo la stringa JSON
      ↪ valida, senza usare Markdown, testo descrittivo o la
      ↪ parola \json".
11     '''
12 )

```

Di conseguenza il modello restituisce una stringa JSON del seguente tipo.

```

1  [{
2      "type": "scontrino",
3      "datetime": "01-04-2025 14:59",
4      "total": 22.0,
5      "category": "ristorazione",
6      "is_electronic_payment": true
7  },
8  {
9      "type": "ricevuta pos",
10     "datetime": "01-04-2025 15:00",
11     "total": 22.0,
12     "category": "ristorazione",
13     "is_electronic_payment": true
14  },
15  {
16     "type": "altro",
17     "datetime": "02-04-2025 02:54",
18     "total": null,
19     "category": "rapporto d'intervento",
20     "is_electronic_payment": false
21  }]

```

Questa nuova struttura degli oggetti di tipo Scontrino ha consentito al modulo di matching di aggiornarsi e lavorare nel seguente modo:

1. Per ciascuno scontrino si cerca un match con una ricevuta, controllando il totale e la differenza nell'orario.
2. Nel caso in cui venga trovato un match, lo scontrino viene inserito tra i deducibili. In caso contrario si verifica comunque se l'attributo 'pagamento elettronico' sia impostato su vero, poiché ciò indicherebbe l'assenza di una ricevuta, ma la presenza dell'indicazione di un pagamento tracciabile direttamente sullo scontrino.
3. Ogni documento di tipologia 'altro' viene aggiunto alla lista dei deducibili solo nel caso in cui 'pagamento elettronico' è vero e la categoria sia compresa tra quelle previste dalla Legge di Bilancio.

Altri errori di valutazione commessi dal modello, anche se in percentuale molto inferiore rispetto ai precedenti, riguardavano la lettura sbagliata di alcuni dati presenti sugli scontrini. Questo avveniva soprattutto per immagini di bassa qualità, molte delle quali erano state compresse prima dell'invio oppure non erano correttamente orientate. Per ovviare a queste tipologie di errori ho provato l'inserimento di una fase di preprocessing all'interno del modulo di acquisizione delle immagini. In questo modo, prima dell'ottenimento della lista di immagini da inviare in input al modello, a queste veniva aumentato il contrasto e diminuito il rumore tramite librerie python ad hoc. Purtroppo però, a causa della natura troppo eterogenea delle immagini, alcune scattate con il telefono, altre ottenute tramite scansione, in condizioni di luce molto variabili, anche l'efficacia del preprocessing variava molto, migliorando la qualità di lettura in alcune e peggiorandola in altre. Per questo motivo ho deciso di rimuovere tale modifica notando un peggioramento complessivo nell'attività di estrapolazione dei dati. Lo stesso è avvenuto con il tentativo di rotazione automatica delle immagini, al fine di garantire l'orientamento giusto prima dell'invio al modello. Per farlo, ho inserito del codice che ruotasse ciascun documento in tutte le direzioni e utilizzasse l'OCR Tesseract per leggerne il contenuto. La direzione corretta veniva identificata come quella in cui si otteneva il maggior numero di informazioni sensate. Anche qui l'algoritmo funzionava correttamente in alcuni casi ma ruotava immagini già dritte in altri. Un'alternativa sarebbe stata quella di implementare il tutto con delle chiamate a GPT-4o, che avrebbe facilmente identificato il corretto orientamento del documento, ma avrebbe d'altra parte comportato un aumento dei costi e dei tempi d'esecuzione. Per via del numero molto limitato di errori di questo tipo ho deciso dunque di non adottare questa soluzione.

La metrica successiva, quella sul tempo di esecuzione, aveva calcolato un totale di 2350 secondi (4 secondi per immagine circa), corrispondenti a quasi 40

minuti. Questo risultato, seppur all'apparenza negativo, non rappresentava un grande limite nella riuscita del progetto, visto che il sistema era stato progettato per essere eseguito in background, senza nessuna necessità di intervento umano, e confrontato con le settimane di lavoro necessarie per svolgere manualmente il lavoro si trattava di un successo. Nonostante ciò, ho deciso comunque di analizzare nel dettaglio il codice al fine di trovare punti critici in cui poter operare ottimizzazioni in termini di costo computazionale e tempo di esecuzione in generale. In questo modo ho individuato due parti che influenzavano maggiormente il tempo di esecuzione. Una di queste corrispondeva al punto in cui venivano effettuate le chiamate API al modello in maniera sequenziale. Trattandosi infatti di elaborazione cloud di immagini tramite un LLM, ciascuna chiamata all'API richiede inevitabilmente diversi secondi, il che si traduce in un alto tempo di esecuzione se il lavoro deve essere svolto un elevato numero di volte. La soluzione che ho pensato e successivamente implementato per risolvere questo problema è stata quella di effettuare le chiamate API in maniera parallela, sfruttando i thread. In questo modo non occorre attendere l'arrivo di una risposta per effettuare la richiesta successiva, e gli unici limiti nelle chiamate al modello svolte in parallelo erano dati dal numero massimo di workers possibili per la classe `ThreadPoolExecutor` e dai Rate e Concurrency Limits imposti da OpenAI. L'altro punto di forte incidenza sul costo computazionale era quello della ricerca di una ricevuta di pagamento per ogni scontrino. Qui sono riuscito ad ottimizzare il tutto inserendo le ricevute all'interno di un dizionario, in cui ciascun importo totale funge da chiave e il valore associato è una lista contenente tutte le ricevute con quell'importo, ordinate per orario. In questo modo, per ogni scontrino, viene utilizzato l'attributo 'totale' come chiave per accedere al dizionario delle ricevute, e poi si scorre la lista per trovare una corrispondenza in termini di datetime. Il costo computazionale pessimo della ricerca rimane $O(n^2)$, ma quello medio è molto più basso, tendente a $O(n)$ nel caso in cui i totali delle ricevute siano tutti differenti tra loro, cosa comunque molto comune nella realtà.

L'ultima metrica rimasta da analizzare era quella del costo da sostenere per l'esecuzione del programma. Ho ricavato tale dato attraverso lo sviluppo di codice che svolgeva il conteggio dei token totali utilizzati per l'input e l'output. Moltiplicando questi numeri con il costo per token corrispondente al modello GPT-4o era possibile ottenere con precisione la spesa totale, verificabile anche tramite la dashboard di OpenAI in mano al responsabile Christian. Per un dataset di 600 immagini sono stati conteggiati i seguenti token:

- Token di input: 807448.
- Token di output: 53337.

Il costo complessivo risultava dunque €2.55, circa 0.004 euro per immagine, assolutamente accettabile e conveniente in confronto a quello necessario per l'analisi manuale.

Una volta implementate le modifiche necessarie alla rimozione degli errori individuati e all'ottimizzazione del tempo di esecuzione ho deciso di effettuare un secondo test sullo stesso dataset per verificare l'effettivo miglioramento. Di seguito sono riportate le metriche rilevate:

- Degli 853 scontrini detraibili presenti ne sono stati individuati 837, un'accuratezza quindi del 98.1%.
- 16 falsi negativi (scontrini detraibili identificati dal sistema come non detraibili), causati tutti da una scarsa leggibilità dei documenti forniti in input. Si potrebbe tentare di correggerli tramite preprocessing più avanzato e/o fine tuning, ma per il momento, in accordo con il responsabile, si è scelto di operare tramite analisi manuale l'identificazione degli eventuali falsi negativi presenti. Questa consiste nella ricerca degli errori all'interno della lista degli scontrini riconosciuti dal programma come non detraibili, solitamente un 10% del totale, il che si traduce nella riduzione del lavoro manuale del 90%, ovvero da un paio di settimane a un giorno circa.
- Il tempo di esecuzione è diventato di 330 secondi, ovvero poco più di 5 minuti, un ottimo risultato rispetto ai 40 minuti del test precedente.
- Il costo totale di esecuzione si è confermato simile a quello precedentemente calcolato, intorno ai €2.50.

Eseguito l'ultimo test e revisionate le metriche con il responsabile, abbiamo deciso di concludere la fase di sviluppo soddisfatti dei risultati ottenuti. Ho passato dunque gli ultimi giorni a creare il file README e ad aggiungere documentazione al codice per facilitarne l'esecuzione e la manutenzione, dopodichè ho presentato e consegnato il progetto ai dipendenti responsabili del gestionale degli scontrini.

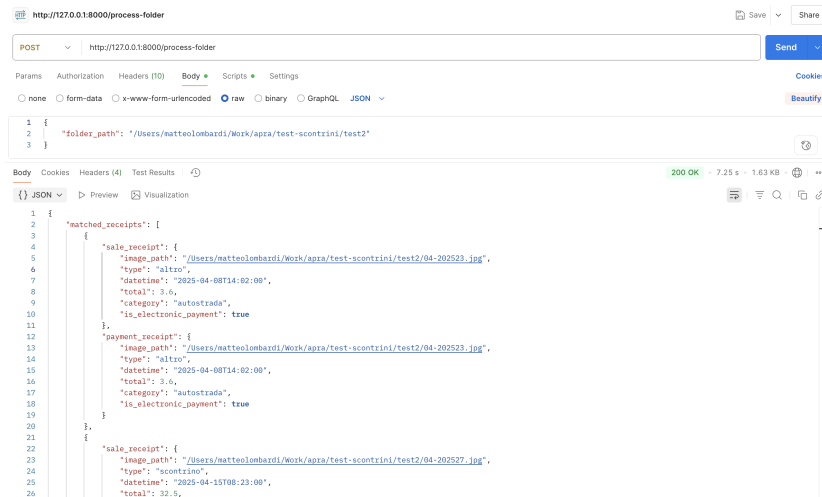
```
1 # AI Receipt Analyzer
2
3 ## Descrizione
4 Sistema automatizzato per l'estrazione e l'analisi dei dati da
5   ↳ immagini di scontrini e ricevute POS, sviluppato in risposta
6   ↳ alla legge di Bilancio 2025.
7
8 ## Requisiti
9 - Requisiti app
10 - python
```

```

9     - pip
10    - pdf2image
11    - openai
12    - fastapi
13    - uvicorn
14    - Requisiti client
15    - python
16    - pip
17    - requests
18
19    ## Installazione
20    1. Clona il repository:
21        ```bash
22        git clone https://github.com/ApraRnD/ai-receipt-analyzer.git
23        cd ai-receipt-analyzer
24        ```
25    2. Installa le dipendenze:
26        - App
27            ```bash
28            pip install -r app-requirements.txt
29            ```
30        - Client
31            ```bash
32            pip install -r client-requirements.txt
33            ```
34
35    ## Utilizzo
36    1. Prepara le immagini o i PDF da elaborare, cercando di adottare
37        ↳ le seguenti pratiche per una maggiore precisione
38        ↳ nell'analisi:
39        - Orienta le immagini nel verso giusto.
40        - Preferisci foto non compresse alle scansioni.
41    2. Esegui il progetto in uno dei due modi seguenti:
42        - In locale
43            ```bash
44            python -m app.main [image_folder_path] [output_folder_path]
45            ```
46        - Client - Server
47            1. Lato server:
48                ```bash
49                uvicorn app.service:app --reload
50                ```
51            2. Lato client:
52                ```bash
53                python -m client.main [server_address] [server_port]
54                ↳ [image_folder_path] [output_folder_path]
55                ```

```

Di seguito è riportato un esempio di chiamata API all'applicativo.



Client - Invio richiesta POST all'endpoint /process-folder

Request:

POST http://127.0.0.1:8000/process-folder
Header: {"accept": "application/json", "content-type": "application/json"}
Body: {"folder_path": [folder_path]}

Server - Elabora le immagini nella cartella specificata

Response (200 OK):

```
{
  "matched_receipts": [
    {
      "sale_receipt": {...}, "payment_receipt": {...},
      "sale_receipt": {...}, "payment_receipt": {...}
    },
    {
      "sale_receipt": {...}, "payment_receipt": {...},
      "sale_receipt": {...}, "payment_receipt": {...}
    }
  ],
  "not_matched_receipts": [
    {
      "sale_receipt": {...}, "payment_receipt": {...},
      "sale_receipt": {...}, "payment_receipt": {...}
    },
    {
      "sale_receipt": {...}, "payment_receipt": {...},
      "sale_receipt": {...}, "payment_receipt": {...}
    }
  ]
}
```

2.8 Costi di esecuzione e scalabilità

Nella sezione precedente è stata descritta la modalità con cui sono stati ottenuti i miglioramenti in termini di performance dell'applicativo. In questa sezione si analizzano più nel dettaglio i risultati raggiunti, con particolare attenzione alla scalabilità complessiva.

Innanzitutto la fase di test e ottimizzazione ha permesso di ottenere i seguenti progressi.

Metrica	Valore Pre-Ottimizzazione	Valore Post-Ottimizzazione
Tempo totale (600 immagini)	2350s (39.2 min)	330s (5.5 min)
Latenza media per immagine	3.92s	0.55s
Throughput	15.3 img/min	109.1 img/min
Accuratezza	82%	98.1%

Tabella 1: Confronto delle prestazioni

Tramite un'analisi più approfondita è possibile discutere anche dei consumi del sistema in termini di CPU e memoria. Dal punto di vista computazionale la maggior parte del carico di lavoro risiede nell'utilizzo del modello LLM tramite cloud. Ciò non consente il controllo diretto del consumo delle risorse e comporta inoltre una latenza e un tempo di risposta variabili. La CPU locale invece viene principalmente utilizzata per le operazioni di gestione file e logica di matching, con un utilizzo contenuto e scalabile per via dei bassi costi computazionali medi adottati.

Per quanto riguarda la memoria RAM, questa viene utilizzata principalmente per mantenere i dati estratti da tutti gli scontrini. Il consumo è dunque lineare rispetto al numero di scontrini e, in generale, rimane basso (nell'ordine di poche centinaia di MB).

L'applicativo è stato realizzato per elaborare batch di immagini in maniera asincrona e parallela, garantendo un'ottima scalabilità orizzontale. Tuttavia quest'ultima è limitata da alcuni fattori non trascurabili:

- Limiti API: I rate limit e la concorrenza massima imposta da OpenAI non consentono un numero troppo elevato di richieste simultanee.
- Costo lineare: L'aumento del numero di immagini in input comporta un incremento proporzionale della spesa economica, essendo questa legata al numero di token elaborati.
- Ricerca matching: Anche l'algoritmo di matching prevede un costo computazionale medio $O(n)$, che potrebbe diventare oneroso per input di dimensioni elevate.

Questi vincoli, per i motivi visti in precedenza, non rappresentano una limitazione nell'utilizzo dell'applicativo, impattando in maniera minima su dataset di dimensioni contenute. Tuttavia, per gestire una probabile crescita futura delle dimensioni degli input, si potrebbe considerare l'adozione di alcune migliorie alla

scalabilità complessiva. Tra queste, l'utilizzo di un'architettura distribuita per bilanciare il carico di lavoro su più istanze del microservizio, l'implementazione di pre-filtri per ridurre il numero di chiamate al modello AI e l'utilizzo di modelli LLM locali, qualora siano giustificati dalle dimensioni e dai costi.

3 Conclusione

L'esperienza di tirocinio presso Apra S.p.A. si è rivelata estremamente utile e formativa, permettendomi di acquisire competenze su alcuni aspetti dell'informatica che ritengo di grande importanza e facendomi comprendere molte dinamiche di sviluppo in un contesto aziendale.

Il sistema sviluppato ha raggiunto e superato gli obiettivi prefissati, arrivando ad un'accuratezza del 98.1% e permettendo quindi di automatizzare un processo manuale di circa il 90%. Il sistema garantisce questi risultati con un costo mensile inferiore ai 10 euro e l'impiego di circa 15 minuti di tempo per il processamento del numero di scontrini che si prevede di ricevere mediamente in un mese. Nonostante i buoni risultati raggiunti, il progetto può essere perfezionato nell'accuratezza e potrà richiedere interventi di manutenzione per implementare nuove tecnologie e/o risolvere futuri problemi. Per questo sin da subito ho compreso l'importanza dell'utilizzo di un'architettura modulare, fondamentale in sistemi di utilizzo aziendale.

Oltre ai risultati portati all'azienda, questo tirocinio mi ha permesso di acquisire nuove competenze. L'implementazione di modelli di intelligenza artificiale all'interno di un programma era una tecnica che desideravo sperimentare da tempo e sono certo che mi sarà utile in futuro, vista la diffusione sempre più ampia di questo tipo di strumenti. Inoltre, ho avuto modo di ampliare la mia conoscenza di Python, un linguaggio che fino a quel momento avevo affrontato solo dal punto di vista teorico e che non avevo mai impiegato nello sviluppo di un'applicazione concreta. Ho compreso anche come i dettagli siano fondamentali nello sviluppo di un software e come la capacità di migliorare un sistema già funzionante costituisca la vera differenza che distingue chi scrive solo codice da chi progetta e migliora soluzioni software tramite un approccio ingegneristico. Alcune nozioni apprese durante specifici corsi universitari si sono rivelate fondamentali per applicare tecniche ed euristiche che hanno portato a miglioramenti significativi tra test condotti a breve distanza l'uno dall'altro.

L'inserimento all'interno di un contesto aziendale mi ha permesso di capire l'importanza di una buona comunicazione con i colleghi e collaboratori, dalla discussione formale degli obiettivi alla documentazione accurata del progetto. D'altra parte, ho compreso anche l'importanza dell'autonomia operativa, fondamentale per ridurre la dipendenza e sviluppare capacità di problem solving e capacità decisionale.

In conclusione, questa esperienza di tirocinio ha superato le mie aspettative, offrendomi la possibilità di lavorare su un progetto innovativo con un impatto concreto sui processi aziendali. La modalità di apprendimento adottata si è rivelata molto efficace, costituendo una connessione diretta tra la formazione accademica e il mondo del lavoro.